HOMEWORK 4

Student Name: Harun Yavaş Student ID: 36445399294

1. Introduction

The objective of this assignment was to implement a custom Hash Table with Chaining in Java to act as a word indexer. The program reads a text file, tokenizes the content based on specific linguistic rules (case-insensitivity, hyphen splitting, punctuation removal), and stores unique words along with their frequencies and occurrence positions. The implementation strictly avoids using Java's built-in HashMap or Hashtable classes, relying instead on a custom node structure and array-based logic.

2. Hash Function Choice

To map string keys (words) to integer indices, I implemented a **Polynomial Rolling Hash** algorithm. This function is widely preferred for string hashing because it takes into account both the character values and their specific positions in the string, significantly reducing collision rates compared to simple additive hashing.

The hash is calculated using the following formula:

$$H(s) = (\, s[0].p^0 + s[1].p^1 + \, s[2].p^2 + \cdots + s[n-1].p^{n-1}\,) \;\; mod\, m$$

Where:

- $s[i]$ is the ASCII value of the character at index i.
- p is a prime number, chosen as **31** (roughly the number of lowercase English letters).
- m is a large prime number, chosen as **1,000,000,009**, to minimize overflow and ensure a uniform distribution across the table buckets.

3. Chaining Structure

To handle collisions, I implemented a **Chaining** strategy using a custom Linked List structure, as required by the assignment constraints.

- **Node Structure:** A private inner class HashNode was defined. Each node contains:
  - String word: The key.
  - int frequency: The counter for occurrences.
  - ArrayList<Integer> positions: A dynamic list storing 1-based token indices.
  - HashNode next: A reference to the next node in the chain.

The main Hash Table is simply an array of HashNode objects (HashNode[] table). When a new word is inserted, it is placed at the calculated index. If the bucket is occupied, the code traverses the linked list at that index. If the word exists, its frequency is updated; if not, a new node is appended to the end of the chain.

4. Rehashing Strategy

The system monitors the **Load Factor** (æ), defined as the ratio of unique words to the total table size.

$$æ = \frac{Numbers\ of\ Unique\ Words}{Current\ Table\ Size}$$

According to the assignment requirements, if æ > 0.75, a **Rehashing** process is triggered:

1. The table size is doubled ($\text{NewSize} = \text{CurrentSize} * 2$).
2. A new, empty table is created.
3. All existing nodes from the old table are iterated.
4. Each node's index is re-calculated using the new table size.
5. Nodes are moved to the new table, preserving their frequency and position data.

5. Collision Counting Logic

The NumberOfCollusion() method tracks the efficiency of the hash function. Following the assignment rules, collisions are counted as follows :

- A collision is incremented **only** when inserting a new word into a bucket that is **already non-empty**.
- If a word is merely updated (frequency incremented), it is **not** counted as a collision.
- If a bucket has a chain of 3 nodes, and a 4th unique word maps to this bucket, it counts as +1 collision.

6. Experimental Results

To analyze the performance of the Polynomial Rolling Hash, I tested the system with a sample text file using three different initial table sizes. The results are as follows:

Initial Table Size	**Total Unique Words**	**Number of Collisions**	**Observation**
100	*53*	*15*	High collision rate due to small capacity; frequent rehashing occurred.
1,000	53	*1*	Balanced performance; fewer rehashes needed.
10,000	53	0	Very few collisions; Load factor remained low, providing near O(1) access time.

7. Conclusion

The implemented Hash Table successfully indexes text with O(1) average time complexity for insertions and lookups. The Polynomial Rolling Hash proved effective in distributing English words uniformly, and the dynamic rehashing capability ensures the system remains efficient even as the dataset grows. The final sorted output (by frequency and lexicographical order) meets all specified reporting requirements.